Sri Lanka Institute of Information Technology



IT2140 - Database Design and Development

2 Year 1 Semester

Assignment 01

Group: 2026-Y2-S1-MLB-B2G2-07

Enhanced Entity Relationship Diagram

StreamHub, an online video browsing and library system

Student ID	Name
IT25101967	Jayalath E.P.A.P.
IT25102896	Weerasinghe W.M.V.K.
IT25103985	Chandrasena H.L.L.S.
IT25101962	Nawarathna N.S.D.
IT25104019	Kawdawatta P.M.A.I.D.
IT25101060	Rajapaksha R.M.H.N.

1. System purpose

StreamHub is a web based video browsing system that gives users a single place to store and organise their own videos while also being able to engage socially around content shared by others. It works like a personal video library where users can upload and manage their own collection, but also includes social features so users can interact with videos uploaded by other people. The platform is open to the public and anyone can create an account.

2. Users

The system supports four types of users.

Role	Account required	Capabilities
Guest	No	Browse and watch public videos, and read comments. Attempting to like, comment, upload, or perform any other protected action opens the register or login screen.
Registered user	Yes, default	Full capability account. Upload videos, edit or delete their own comments, reply to comments one level deep, like or unlike videos, create and manage playlists, report content, and manage their own profile.
Content moderator	Yes, assigned	Assigned manually by an administrator, not selectable at registration. Can delete any video, comment, or playlist, and review and act on reports.
Administrator	Yes, assigned	A superset of the content moderator role. Also has full category management, user role management, and access to system analytics.

3. Functional requirements

The system is organized into six major functions, each assigned to a different group member and each owning its own data and its own part of the interface.

3.1 Video upload and management

- Users can upload a video with a title, description, category, tags, and thumbnail.
- Users can edit the metadata of their own videos and delete their own videos.
- Each video has a visibility setting of public, private, or unlisted.
- The system provides an in app video player, served through byte range streaming so users can seek within a video.
- Deleting a video also removes its related comments, like, watch history entries, and playlist entries.

3.2 Category management

- Only administrators can create, edit, or delete categories.
- Categories are a flat list, with no subcategories.
- Every video belongs to exactly one category, and users can browse or filter videos by category.

3.3 Playlist and content organization

- Only registered users can create playlists.
- A playlist can contain a mix of the owner's own videos and other users' public videos, stored as references.
- Playlists have their own visibility setting, either public or private.
- Users can reorder the videos within their own playlist.

3.4 Social interaction

- Users can like or unlike a video.
- Users can comment on a video and edit or delete their own comments.
- Replies to a comment are supported.
- Comments apply to videos only.
- Like counts and comments are visible to everyone, including guests, but guests cannot interact.

3.5 Report and moderation management

- Users can report a video, a comment, a playlist, or a user profile.
- A report includes a reason chosen from a fixed list, plus an optional free text description.
- A report moves through a status lifecycle of pending, reviewed, and then resolved or dismissed.
- For reports on a video, comment, or playlist, a moderator can take direct action such as deleting or hiding the content.
- For reports on a user profile, a moderator can only escalate the report to an administrator or dismiss it, and cannot act on the account directly.

3.6 Watch history and recommendation management

- The system logs which video a user watches, together with a timestamp and watch progress.
- Users can view and clear their own watch history.
- The system recommends videos based on the categories a user watch most often.

4. Non functional requirements

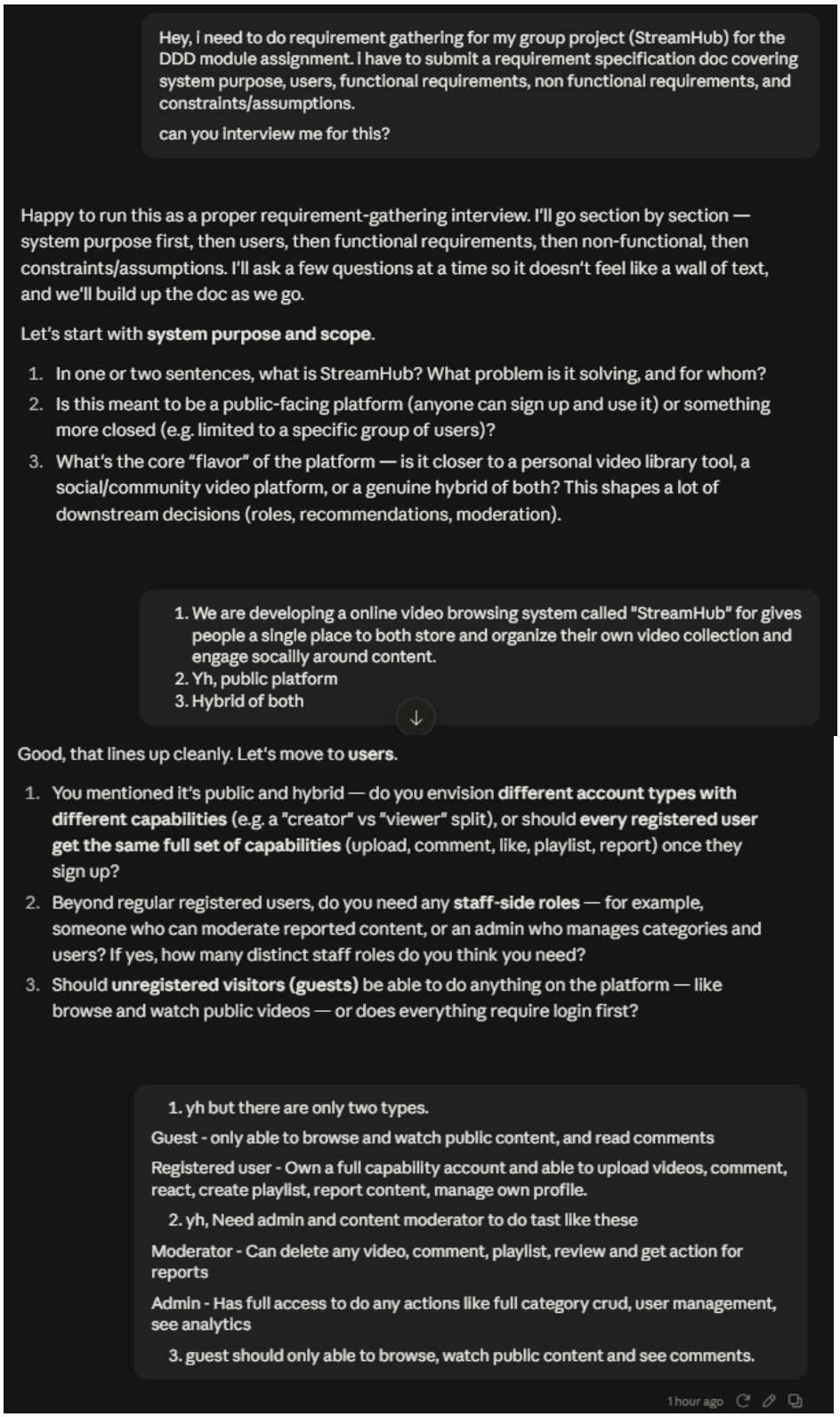
- Performance: video playback should start within about two seconds of the user selecting a video.
- Security: passwords are hashed with BCrypt, and authentication uses JSON web tokens.
- Usability: the interface should be usable and responsive on both desktop and mobile screens.
- Scalability and extensibility: the database schema and the API are structured so new categories or features can be added later without a redesign.
- Data integrity: input is validated on both the frontend, for a better user experience, and the backend, as the real enforcement point, so invalid data cannot bypass checks by calling the API directly.
- Maintainability: the backend is organized by feature, so each major function owns its own controller, service, repository, and entity, and can be changed without affecting the others.

5. Constraints and assumptions

- The technology stack is fixed: Spring Boot with Java 17 for the backend, React for the frontend, Microsoft SQL Server accessed through Spring Data JPA, Spring Security with JWT for authentication, and local file storage rather than cloud storage.
- The team has six members and a fixed semester timeline.
- Despite the academic time constraints, the system is intended to be a fully functional, logically consistent system, not a stripped down prototype.

6. Evidence of requirement gathering

The requirements documented above were gathered through a structured conversation with a Claude AI. Screenshots of that conversation are included below as evidence.



Perfect, this all matches what I'd expect for a platform like this. Quick follow-ups to lock in the role model precisely:

1. You said "Guest" and "Registered user" as the two signup-facing types — so there's **no separate "Creator" account type at registration**, correct? Everyone who registers gets the same full capability set (upload/comment/like/playlist/report), and there's no split where some registered users can't upload?
2. For **Moderator** and **Admin** — can a new user ever select these roles at registration, or are they **assigned manually** (e.g. by an existing Admin) after the account already exists as a normal Registered User?
3. Is **Moderator** a completely separate role from Admin, or is Admin a **superset** of Moderator (i.e. Admin can do everything Moderator can, plus more)? You listed Admin's powers as category CRUD, user management, and analytics — does that mean Admin **cannot** directly delete videos/comments/reports (moderator-only job), or can Admin do that too?

1. guest has no account, he is just a random visitor. no separate account types and yh, thats correct about the registered user.
2. they assigned manually by existing admin
3. yh admin is a superset of moderator

Excellent, that's a clean, standard role hierarchy. So to summarize what we've locked in for **Users**:

- **Guest** — no account, unauthenticated visitor; browse public videos, watch public videos, read comments only
- **Registered User** — the only account type available at signup; full capability: upload, comment, like/react, playlist, report, manage own profile
- **Content Moderator** — assigned manually by an Admin; delete any video/comment/playlist, review and act on reports
- **Administrator** — superset of Moderator; plus category CRUD, user management (including assigning Moderator role), and analytics access

Now let's move into **Functional Requirements** — this is usually the biggest section. I know StreamHub has six major functions split across your team. Let's go one at a time so each is properly scoped.

Function 1: Video Upload & Management

1. What are the core operations here — upload, edit metadata, delete, and anything else (e.g. replace thumbnail, set video visibility public/private)?
2. What metadata does a video need at minimum — title, description, category, tags, thumbnail, visibility status, anything else?
3. Does this function also own **video playback** (the in-app player / streaming), or is that considered a separate concern?

1 hour ago

1. yh those are the core task.
2. yh, thats what need to include for metadata
3. yh this own video playback

Good, that matches cleanly. Let's nail down a couple of specifics for **Function 1** before moving on:

1. For **visibility**, is it a simple binary (Public / Private), or do you want a third state like "Unlisted" (viewable via link but not shown in public listings)?
2. For **playback**, since this is a "video browsing system" with your own file storage rather than embedding YouTube/Vimeo — should the video files be served with **byte-range streaming support** (so users can seek/scrub through the video, and it doesn't have to download fully before playing)? This matters for the non-functional requirements section too.
3. Does **deleting a video** need to cascade — i.e. should it also remove related comments, likes, watch history entries, and playlist entries pointing to it? Or should deletion be blocked/restricted if the video has related content?

1 hour ago

1. yh need to add unlisted state too.
2. yh, its better to served with byte range streaming
3. yh, it should be cascade

Good, all consistent and sensible. Let's move to **Function 2: Category Management**.

1. Who can create/edit/delete categories — is this Admin-only (matching what we established earlier), or can Registered Users also suggest/create their own categories?
2. Is a video restricted to **exactly one category**, or can a video belong to **multiple categories** at once?
3. Do you need **subcategories** (nested hierarchy, e.g. "Music" → "Live Performances"), or is a **flat list of categories** enough for this system?

1. Its admin only
2. restricted for one category
3. flat list

Clean and simple — that keeps the schema straightforward too (single FK on Video, no join table needed for categories).

Now **Function 3: Playlist & Content Organization**.

1. Who can create playlists — only Registered Users, and only containing their **own** videos, or can a playlist mix in other people's public videos too?
2. Should a playlist have its own **visibility setting** (Public/Private), similar to videos — so a user could keep a playlist just for themselves, or share it?
3. Do you need the ability to **reorder videos within a playlist** (custom order), or is insertion order fine?

1. only Registered Users, there should be two options, first can create playlist with their own videos, second can create playlist mix in other people's public videos
2. yh
3. yh

Good, that's more flexible than a simple "own videos only" model — makes sense for a hybrid personal-library + social platform. Let's pin down the details:

1. When a playlist mixes in other people's public videos, does the playlist just store references (video IDs) — so if the original uploader later makes that video private or deletes it, it simply drops out of / breaks in the playlist? Or do you need some other handling?
2. For **reordering** — do you want a simple "move up/move down" per video, or drag-and-drop style reordering (functionally the same on the backend, just confirming so I record the requirement correctly)?
3. Can the same video appear in **multiple different playlists** owned by the same user, or even across different users' playlists (since they're just references)?

1. yh just store references
2. move up and down
3. it better to allow to add mult[illegible] playlists, theire can be different types of vibe in one video

Good, that's consistent with a reference-based many-to-many design — one video can sit in many playlists, and a playlist can mix owned + others' public videos.

That closes out **Function 3**. Moving to **Function 4: Social Interaction**.

1. For "react," is it a simple **Like** (like/unlike toggle), or do you want multiple reaction types (like, dislike, love, etc.)?
2. For **comments** — can a user **edit or delete their own comment** after posting? And can they be **nested/threaded** (replies to a comment), or is it a flat comment list per video?
3. Can Guests **see** comments (already confirmed yes) but obviously not post them — does a Guest trying to comment/like get **blocked silently**, or should they see a **prompt to register/login** (like YouTube's "sign in to like this video" behavior)?

1. just Like and Unlike toggle.
2. yh they can edit or delete their own comment, and able to reply for comment even its their own comment
3. only can see comments, if he try to comment /like should trigger register/login

Noted — that adds one more piece to the Comment structure (self-referencing parent comment for replies), and confirms the guest-gate behavior matches the YouTube-style pattern.

Quick follow-ups on **Function 4**:

1. For **replies to comments** — do you want just **one level of nesting** (a comment and its direct replies, like most simple systems), or **infinite nesting** (replies to replies to replies)? One level is usually enough and much easier to display cleanly in UI.
2. Should **Like counts be visible to everyone** (including guests), even though only registered users can actually click it?
3. Does a video's **comment section only apply to videos**, or should users also be able to comment on other things like **playlists**? (Just confirming scope — some platforms only allow comments on videos.)

39 minutes ago

1. one level of nesting is enough
2. yh
3. only apply to videos

Good, that's the standard and cleanest approach — keeps the Comment table simple (one nullable parent_comment_id, one level deep) and Like/comment scope restricted to Video only.

Moving to **Function 5: Report & Moderation Management**.

1. What can be reported — just Videos, or also Comments, Playlists, and User Profiles?
2. When a user submits a report, do they pick from a **fixed list of reason categories** (e.g. spam, harassment, copyright, inappropriate content), or is it a free-text field?
3. What's the **status lifecycle** of a report — e.g. Pending → Reviewed → Resolved/Dismissed? And earlier you mentioned Moderators "review and act on reports" — does "acting" mean they take the moderation action (delete/hide the content) directly from the report, or is that separate?

1. should able to report any of those
2. reasons categories + freetext fiels as optional
3. Pending → Reviewed → Resolved/Dismissed, yh they take the moderation action

31 minutes ago

Good — a fixed reason category plus an optional free-text field is a solid pattern (structured for filtering/reporting stats, free-text for detail), and a clear three-stage lifecycle with the moderator acting directly.

One clarification on **Function 5** before moving on:

1. You said Moderators can act on reports for Video/Comment/Playlist (delete/hide directly) — but what about reports on **User Profiles**? A moderator can't really "delete" a user account (that's more of an Admin-level action, e.g. ban/suspend). So for a Profile report, should the moderator's options be limited to something like **escalate to Admin** or **dismiss**, rather than taking direct action themselves?

Once that's confirmed, let's move to the last one — **Function 6: Watch History & Recommendation Management.**

1. What exactly gets logged in watch history — just "video X was watched by user Y at time Z", or do you also need things like **watch duration/progress** (e.g. for a "continue watching" feature)?
2. For **recommendations** — what's the strategy: content-based (recommend videos in categories the user watches most), collaborative (based on what similar users watched), trending/popularity-based, or some mix?
3. Should users be able to **view and clear their own watch history** (privacy control), similar to YouTube's history page?

31 minutes ago

1. for profile report limited to escalate to Admin

function 6

1. its better to track watch duration too
2. recommend videos in categories the user watches most
3. yh. users be able to view and clear their own watch history

Good, that all locks in cleanly. Category-affinity based recommendations is also a clean fit for a design pattern later (Strategy pattern — swap out the recommendation algorithm without touching the rest of the system).

That completes all **six major functions.** Let's move to **Non-Functional Requirements** — these describe *how well* the system performs rather than *what* it does.

1. **Performance** — any specific expectations, like page load times, or how many concurrent users the system should reasonably handle (even a rough target, since this is an academic project not production scale)?
2. **Security** — beyond JWT-based authentication, do you want to call out specific things like password hashing, input validation against SQL injection, or role-based access control (RBAC) explicitly enforced at the backend?
3. **Usability** — any specific requirement here, like the UI should be responsive across desktop/mobile, or consistent design language across all six functions (since six people are building six parts)?

1. video playback should start within roughly 2 seconds
2. passwords should hashed with BCrypt
3. Interface should be usable, and its better to be responsive across desktop/mobile

Good — BCrypt is the standard pairing with Spring Security anyway, so that's consistent with the JWT auth setup. Let's round out the NFRs with a couple more categories:

1. **Scalability/Reliability** — do you want to state anything about the database connection needing to be stable/reconnect cleanly, or is that more of a testing concern than a formal NFR for this doc?
2. **Data integrity** — should I record something like "all critical operations must be validated both on the frontend (UX) and backend (Bean Validation / enforced constraints), so invalid data can never bypass validation by calling the API directly"?
3. **Maintainability** — since this is a feature-based, six-person codebase, do you want an NFR stating something like "each function's code should be modular and independently maintainable, minimizing coupling between functions" — useful to mention since it justifies your architecture choices in the doc?

1. database schema and API are should be structured, so adding more categories, users,...etc later does not require a redesign
2. yh
3. backend isshould be organized by feature, with each major function owning its own controller, service, repository, and entity, so individual functions can be modified without breaking unrelated modules

Good, that's a strong scalability/extensibility statement and ties in nicely with your feature-based backend structure and single-entity-owner-per-function principle.

Now let's finish the requirements side with **Constraints & Assumptions** — this is typically shorter but important for scoping honestly.

1. **Technology constraint** — should I record that the tech stack is fixed as: Spring Boot (Java 17) backend, React frontend, Microsoft SQL Server via Spring Data JPA, Spring Security + JWT, and local file storage for videos (no cloud storage like AWS S3)?
2. **Team constraint** — is it worth noting that this is a 6-person team project with a fixed timeline (semester-based, defined sprint weeks), which limits scope compared to a commercial platform?
3. **Assumptions** — anything you want to explicitly assume, like "users have a stable internet connection for video upload/playback" or "the system is not expected to handle production-scale traffic (it's an academic prototype)"?

11 minutes ago

1. yh that better
2. yh, we have limited time
3. yh users have a stable internet connection for video upload/playback but this is academic prototype with fully logically and functionally working system. not just a prototype

Good catch, that's an important distinction — noted: it's an **academic project with a fully functional, working system** (not a throwaway proof-of-concept), built under semester time constraints as a team. I'll phrase that carefully in the doc so it doesn't read as "half-finished by design."

That completes all five sections. Here's a quick recap of everything locked in for this session:

System Purpose — Public hybrid platform (personal library + social features)

Users — Guest (no account), Registered User (only signup type, full capability), Content Moderator (assigned by Admin), Administrator (superset of Moderator)

Functional Requirements (6 major functions) — Video Upload & Management (with playback, unlisted state, cascade delete), Category Management (admin-only, flat, one-per-video), Playlist & Content Organization (own + others' public videos, reorder, multi-playlist), Social Interaction (like/unlike, one-level comment replies, edit/delete own comment, guest gate), Report & Moderation (4 reportable types, category+freetext reason, 3-stage lifecycle, profile reports escalate-only), Watch History & Recommendation (duration tracking, category-affinity recommendations, user can clear history)

Non-Functional Requirements — ~2s playback start, BCrypt hashing, responsive UI, extensible schema/API, backend+frontend validation, feature-based modular backend

Constraints/Assumptions — Fixed tech stack, 6-person team/semester timeline, stable internet assumed, fully functional system (not a stripped-down prototype)